```
#include <iostream>
#include <algorithm>
#include <iomanip>
#include <string>
using namespace std;
struct Item {
    int id, weight, value;
};
bool compare(Item a, Item b) {
    return a.value > b.value;
}
void printRow(string label, int row[], int W) {
    cout << left << setw(14) << label << right;
    for (int w = 0; w <= W; w++) cout << setw(4) << row[w];
    cout << endl;
}
int main() {
    int n, W;
    cout << "Enter number of items: ";
    cin >> n;
    Item items[51];
    cout << "Enter weight and value of each item:\n";
    for (int i = 1; i <= n; i++) {
        items[i].id = i;
        cin >> items[i].weight >> items[i].value;
    }
    cout << "Enter knapsack capacity: ";
    cin >> W;
    stable_sort(items + 1, items + n + 1, compare);
    cout << "\nItems sorted by profit (highest first):\n";
    for (int i = 1; i <= n; i++)
        cout << "Item " << items[i].id << ": weight = " << items[i].weight << ", profit = "
                << items[i].value << endl;
    int dp[51][101];
    int cap[101];
    for (int w = 0; w <= W; w++) {
        cap[w] = w;
        dp[0][w] = 0;
    }
    cout << "\nStart (no items considered):\n";
    printRow("Capacity", cap, W);
    printRow("No items", dp[0], W);
    for (int i = 1; i <= n; i++) {
        for (int w = 0; w <= W; w++) {
            if (w == 0)
                dp[i][w] = 0;
            else if (items[i].weight <= w)
                dp[i][w] = max(dp[i - 1][w], items[i].value + dp[i - 1][w - items[i].weight]);
            else
                dp[i][w] = dp[i - 1][w];
        }
        cout << "\nStep " << i << ": Item " << items[i].id << " (weight = " << items[i].weight
                << ", profit = " << items[i].value << ")\n";
        printRow("Capacity", cap, W);
        printRow("After item " + to_string(items[i].id), dp[i], W);
        cout << "Best profit so far = " << dp[i][W] << endl;
        if (i == n) {
            cout << "Remaining items: none\n";
        } else {
            cout << "Remaining items (highest profit first):\n";
            for (int j = i + 1; j <= n; j++)
                cout << "  Item " << items[j].id << ": weight = " << items[j].weight << ", profit = "
                        << items[j].value << endl;
        }
    }
    cout << "\nMaximum value = " << dp[n][W] << endl;
    cout << "\nSelecting items (going back from the last step):\n";
    int w = W, total = 0;
    for (int i = n; i > 0; i--) {
        if (dp[i][w] != dp[i - 1][w]) {
            w -= items[i].weight;
            total += items[i].value;
            cout << "Item " << items[i].id << " selected, remaining capacity = " << w
                    << ", profit so far = " << total << endl;
```

```
        } else {
            cout << "Item " << items[i].id << " not selected\n";
        }
    }
    return 0;
}
```

```
PS C:\Users\fishb\Desktop\Sem 7\DAA> cd "c:\Users\fishb\Desktop\Sem 7\DAA\" ; if ($?) { g++ 4_knapsack_01.cpp -o 4_knapsa
ck_01 } ; if ($?) { .\4_knapsack_01 }
Enter number of items: 4
Enter weight and value of each item:
1 1
3 4
4 5
5 7
Enter knapsack capacity: 7

Items sorted by profit (highest first):
Item 4: weight = 5, profit = 7
Item 3: weight = 4, profit = 5
Item 2: weight = 3, profit = 4
Item 1: weight = 1, profit = 1

Start (no items considered):
Capacity        0   1   2   3   4   5   6   7
No items        0   0   0   0   0   0   0   0

Step 1: Item 4 (weight = 5, profit = 7)
Capacity        0   1   2   3   4   5   6   7
After item 4    0   0   0   0   0   7   7   7
Best profit so far = 7
Remaining items (highest profit first):
  Item 3: weight = 4, profit = 5
  Item 2: weight = 3, profit = 4
  Item 1: weight = 1, profit = 1
```

```
PS C:\Users\fishb\Desktop\Sem 7\DAA> cd "c:\Users\fishb\Desktop\Sem 7\DAA\" ; if ($?) { g++ 4_knapsack_01.cpp -o 4_knapsa
ck_01 } ; if ($?) { .\4_knapsack_01 }

Step 2: Item 3 (weight = 4, profit = 5)
Capacity        0   1   2   3   4   5   6   7
After item 3    0   0   0   0   5   7   7   7
Best profit so far = 7
Remaining items (highest profit first):
  Item 2: weight = 3, profit = 4
  Item 1: weight = 1, profit = 1

Step 3: Item 2 (weight = 3, profit = 4)
Capacity        0   1   2   3   4   5   6   7
After item 2    0   0   0   4   5   7   7   9
Best profit so far = 9
Remaining items (highest profit first):
  Item 1: weight = 1, profit = 1

Step 4: Item 1 (weight = 1, profit = 1)
Capacity        0   1   2   3   4   5   6   7
After item 1    0   1   1   4   5   7   8   9
Best profit so far = 9
Remaining items: none

Maximum value = 9

Selecting items (going back from the last step):
Item 1 not selected
Item 2 selected, remaining capacity = 4, profit so far = 4
Item 3 selected, remaining capacity = 0, profit so far = 9
Item 4 not selected
```